



東南大學

计算机视觉基础大作业

LASA：面向边缘 AI 部署的 MPSoC 层自适应脉动阵列加速器设计

姓 名：	郝嘉恒
学 号：	61523529
学 院：	吴健雄学院
专 业：	电子科学与技术（集成电路方向）
指导老师：	齐志

二〇二六年六月

摘要

边缘端目标检测要求在有限功耗和片上资源下完成低延迟卷积神经网络推理。可编程逻辑与多处理器片上系统平台具有可定制数据通路和软硬件协同优势，适合部署轻量级检测网络。然而，在真实网络端到端运行中，性能并不只取决于乘加阵列规模；DMA 启动、输入特征图重排、权重加载、partial-PSUM 反馈、多轮 K pass 同步和软硬件调度空泡都会降低阵列有效利用率。

本文面向 Xilinx Kria KV260/XCK26 平台，设计并实现了一种用于 YOLOv3-tiny 单尺度链式推理的层自适应脉动阵列加速器 LASA (Layer-Adaptive Systolic Accelerator)。LASA 以 dual-lane INT8 systolic array 为卷积核心，原生支持 1x1/3x3 卷积、INT32 部分和累加、逐通道重量化以及可旁路激活和池化。软件侧基于 Vitis bare-metal runtime 完成层描述、tile 调度、DMA 启动、输出重排和 YOLO decode，形成从 DDR 图像输入到 Conv0–Conv9 输出解析的完整 PS/PL 协同链路。

围绕真实 YOLO 层的非计算开销，本文提出 KCS 三维分块执行模型，并进一步实现三类优化策略：BSD (Batched Streaming Dataflow) 降低 PS/DMA 交互开销，OCRR (On-Chip Reorder and Replay) 将 IFM 布局转换转移到片上缓存和重放逻辑，OPF-P (Overlapped PSUM Feedback and Prefetch) 隐藏多 K pass 中的 partial-PSUM 反馈和下一 pass 准备气泡。本文同时设计了阶段、子阶段和 pass 级性能计数器，用于解释瓶颈迁移和验证优化有效性。

实验结果表明，所设计系统在 KV260 上完成 Conv0–Conv9 batch-chain bit-exact 验证，并在两张 DDR 图像上得到稳定 YOLO decode 结果。最终推荐实现采用 Vivado/Vitis 2022.2、100 MHz 时钟、ROWS=18、COLS=8、COUT_TILE=16 的配置，端到端 DDR demo 延迟约为 288 ms；相较早期约 1.18 s 的链式推理版本，获得约 4.1 倍端到端加速。阶段计数器结果进一步表明，LASA 与 KCS/BSD/OCRR/OPF-P 能够降低数据搬运、IFM 布局转换、partial-PSUM 反馈和 pass 边界调度带来的非计算开销。

关键词：YOLOv3-tiny, MPSoC, FPGA 加速器, LASA, systolic array, 数据流优化, INT8 量化

目录

第一章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.3 本文主要工作.....	2
1.4 论文组织结构.....	3
第二章 背景与相关工作.....	5
2.1 卷积计算与分块表示.....	5
2.2 INT8 量化与后处理.....	6
2.3 Systolic Array 与数据流.....	6
2.4 YOLOv3-tiny 单尺度网络.....	6
2.5 MPSoC 平台与 AXI 接口.....	7
2.6 相关工作比较.....	8
第三章 LASA 层自适应脉动阵列架构.....	9
3.1 系统设计目标与软硬件分工.....	9
3.2 MPSoC 总体结构.....	9
3.3 LASA 数据通路概览.....	10
3.4 层自适应机制.....	10
3.5 链式推理执行流程.....	11
3.6 本章小结.....	12
第四章 面向链式推理的数据流与调度优化.....	13
4.1 KCS 三维分块执行模型.....	13
4.2 BSD: 面向 PS/PL 交互的批量流式协同.....	14
4.3 OCRR: 面向 IFM 布局转换的片上重排与重放.....	15
4.4 OPF-P: 面向 K-pass 边界的反馈重叠与预取.....	15
4.5 优化策略小结.....	16
第五章 运行时集成、验证机制与性能诊断.....	19
5.1 Layer Descriptor 与链式调度.....	19
5.2 AXI-Lite 配置接口.....	19
5.3 正确性验证路径.....	21

5.4	性能计数器设计.....	21
5.5	最终推荐配置.....	21
5.6	探索性 Column-Level PSUM	22
5.7	本章小结	23
第六章	验证与实验结果	25
6.1	实验设置	25
6.2	正确性验证	25
6.3	端到端性能与优化消融.....	26
6.4	阶段计数器分析.....	28
6.5	资源与时序	29
6.6	与相关工作的定性比较.....	29
6.7	讨论与后续工作.....	29
6.8	本章小结	30
参考文献		31

表格索引

表 2.1	Conv0–Conv9 单尺度链路层参数	7
表 4.1	KCS 执行模型与三类优化策略	17
表 5.1	主要运行配置寄存器	20
表 5.2	主要性能计数器类别	22
表 6.1	关键正确性验证结果	26
表 6.2	端到端 DDR demo 优化消融	26
表 6.3	最终 DDR image demo 结果	26
表 6.4	最终版本关键阶段计数器	28
表 6.5	最终推荐构建资源与时序	29
表 6.6	与代表性工作的定性比较	30

插图索引

图 1.1	LASA 系统框架与优化路线	3
图 2.1	YOLOv3-tiny 单尺度检测链路	7
图 3.1	MPSoC 端到端系统结构	10
图 3.2	LASA 数据通路概览	11
图 4.1	BSD、OCRR 与 OPF-P 三类优化策略总览	13
图 4.2	KCS 三维分块执行模型	14
图 4.3	OPF-P 反馈重叠与 pass 级预取时间线	17
图 6.1	端到端 DDR demo 归一化延迟与相对加速比	27
图 6.2	A53 软件基线与最终硬件端到端延迟对比	27
图 6.3	1×1 卷积层原生路径与稀疏 3×3 映射延迟对比	28

主要符号对照表

CNN	卷积神经网络 (Convolutional Neural Network)
IFM	输入特征图 (Input Feature Map)
OFM	输出特征图 (Output Feature Map)
PSUM	部分和 (Partial Sum)
DSP	数字信号处理单元 (Digital Signal Processing slice)
BRAM	块随机存取存储器 (Block RAM)
LUT	查找表 (Look-Up Table)
PE	处理单元 (Processing Element)
FSM	有限状态机 (Finite State Machine)
AXI	高级可扩展接口 (Advanced eXtensible Interface)
DMA	直接存储器访问 (Direct Memory Access)
LASA	层自适应脉动阵列加速器 (Layer-Adaptive Systolic Accelerator)
KCS	K 维、输出通道和空间三维分块执行模型 (K-Cout-Spatial)
BSD	批量化流式数据流 (Batched Streaming Dataflow)
OCRR	片上重排与重放 (On-Chip Reorder and Replay)
OPF-P	重叠式部分和反馈与预取 (Overlapped PSUM Feedback and Prefetch)
K_{total}	展开后输入通道总维度, $K_{\text{total}} = C_{in} \times K_h \times K_w$
K_{TILE}	单次 K 分块处理的输入 lane 数
C_{out}	输出通道数
$\text{COUT}_{\text{TILE}}$	单次计算覆盖的输出通道数
H, W, C	特征图高度、宽度、通道数
K_h, K_w	卷积核高度、宽度
oy, ox	输出特征图空间坐标
GEMM	通用矩阵乘法 (General Matrix Multiply)
FPGA	现场可编程门阵列 (Field Programmable Gate Array)
XCK26	Xilinx Kria K26 核心 FPGA 器件型号
WNS	最差负时序裕量 (Worst Negative Slack)
HWC	高度-宽度-通道 (Height-Width-Channel) 内存布局

第一章 绪论

1.1 研究背景与意义

卷积神经网络 (Convolutional Neural Network, CNN) 已经成为目标检测、图像分类和语义理解等视觉任务的核心计算模型。YOLO 系列检测器将候选框生成和类别预测整合为单阶段推理流程, 在边缘端实时感知任务中具有较好的工程适配性^[1-2]。其中 YOLOv3-tiny 通过减少层数和通道规模降低计算量, 适合在嵌入式平台上部署。

边缘端部署面对的约束与数据中心不同。通用 CPU 的吞吐有限, 嵌入式 GPU 或专用 NPU 虽然性能较高, 但在功耗、可定制性、接口灵活性和可验证性方面并不总是满足定制系统需求。FPGA 与 MPSoC 平台同时包含处理系统 (Processing System, PS) 和可编程逻辑 (Programmable Logic, PL), 能够让软件负责图像输入、调度和后处理, 让硬件承担规则密集的卷积计算。因此, 基于 FPGA 的 CNN 加速器仍然是边缘智能系统中的重要研究方向^[3-4]。

另一方面, 真实检测网络的层结构具有明显异构性。以本文采用的 YOLOv3-tiny Conv0–Conv9 链路为例, 前端层空间尺寸较大且常伴随池化, 后端层空间尺寸较小但通道数显著增加; 网络同时包含 3x3 卷积和 1x1 检测头, 不同层对 activation、pooling、K pass 数量和输出通道分块的需求也不相同。若硬件仅针对单一卷积核或固定后处理路径设计, 则 1x1 层可能被低效映射为稀疏 3x3, 池化和激活需要频繁回退到软件, 或者每类层都需要独立硬件路径, 最终降低资源利用率和系统可扩展性。因此, 本文首先需要构建一套层自适应的卷积执行基础, 使同一套脉动阵列能够通过运行时描述符适配不同层形状和后处理组合。

然而, 仅具备层自适应能力仍不足以保证端到端性能。真实 CNN 网络的加速并不等同于构造一个高峰值 MAC 阵列; 每一层还要经过输入通道展开、输出通道分块、空间 tile 分块和多轮 K pass 累加。若数据搬运、部分和反馈或 pass 边界调度处理不当, 硬件阵列会在等待权重、等待 IFM replay、等待 partial-PSUM drain 或等待 DMA 完成时产生大量空泡。本文关注的核心问题正是: **如何在资源受限的 MPSoC 上, 构建层自适应的卷积加速基础, 并面向真实 YOLOv3-tiny 链式推理降低非计算开销, 使优化效果能够被板级实验和性能计数器解释。**

1.2 国内外研究现状

CNN 加速器的研究大致经历了从算子级流水线到空间阵列架构的发展。Kung 提出的 systolic architecture 通过规则 PE 网络在相邻单元之间传递数据，为矩阵乘和卷积计算提供了高复用的硬件组织方式^[5]。Google TPU 使用大规模 systolic array 获得高吞吐推理性能^[6]。Eyeriss 系列则从能量效率角度系统讨论了 row-stationary 数据流和层级存储复用^[7-8]。Sze 等人的综述对 DNN 加速器的数据流、存储层次和能效权衡进行了系统总结^[9]。

在 FPGA 方向，Qiu 等人较早展示了在嵌入式 FPGA 上部署 CNN 的完整流程^[3]；Angel-Eye 进一步提出从模型到 FPGA 映射的设计流程^[4]；S2A 和 RTL compilation 相关工作关注从高层描述生成 systolic array 或模块化 RTL^[10-11]。针对 YOLOv3-tiny 的轻量化 FPGA 加速器也已有报道^[12]。这些工作说明 FPGA 对 CNN 推理具有可行性，但许多研究更强调模型映射、阵列生成或单层性能，对真实链式推理中的 DMA 交互、IFM 打包、partial-PSUM 反馈和 pass 级调度开销讨论较少。

本文以可复现的 MPSoC 端到端系统为载体，研究真实 YOLO 链式推理中的数据流和调度瓶颈。相比仅报告理论吞吐或单层仿真的工作，本文更强调硬件 RTL、Vitis runtime、外部 golden、板级 batch-chain 验证、DDR 图像输入和性能计数器之间的一致闭环。

图 1.1 给出本文的系统框架与优化路线。论文首先从真实 CNN 链式推理中的数据搬运和调度开销出发，构建 PS/PL 协同的 LASA 卷积执行引擎；随后以 KCS 三维分块模型统一层映射，并通过 BSD、OCRr 和 OPF-P 分别优化 PS/PL 交互、IFM 数据排布和 K-pass 边界空泡；最终通过 golden 验证、DDR image demo 和性能计数器形成板级诊断闭环。

1.3 本文主要工作

围绕上述问题，本文完成的主要工作如下。

1. **提出并实现 LASA 层自适应脉动阵列加速器。** LASA 基于 dual-lane INT8 systolic array，支持 3x3 卷积、原生 1x1 卷积、INT32 partial-PSUM、逐通道重量化、可旁路 activation LUT、可旁路 2x2 maxpool 和 OFM 写回；系统接口采用 AXI-Lite 配置和四路 AXI-Stream/DMA 数据通路，能够与 A53 bare-metal runtime 协同执行 Conv0–Conv9。
2. **提出 KCS 三维分块执行模型。** 本文将不同卷积层统一映射为 K-pass、Cout block 和 spatial tile 三个调度维度。K-pass 适配阵列输入 lane 数，Cout block 适配 dual-lane 输出通道并行度，spatial tile 约束片上缓存容量和 DMA 搬运粒度，为后续

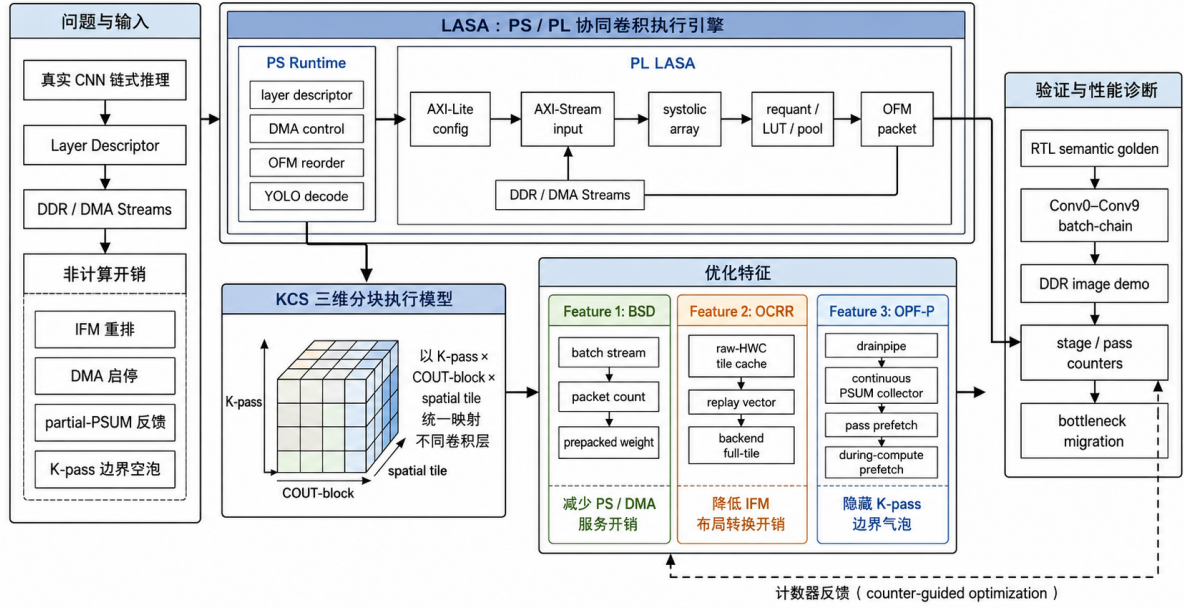


图 1.1 LASA 系统框架与优化路线

数据流优化提供统一执行抽象。

3. 提出面向非计算开销的三类优化策略。面向 PS/PL 交互开销，BSD 通过 batch stream、packet count 和预打包权重减少 DMA 启动与软件服务次数；面向 IFM 布局转换开销，OCRr 通过 raw-HWC tile cache、1x1/3x3 replay 和 backend full-tile 降低软件重排与 tile 边界成本；面向 K-pass 边界气泡，OPF-P 通过流水化 drain、continuous PSUM collector、pass-level prefetch 和 during-compute prefetch 重叠部分和反馈与下一 pass 准备。
4. 建立从 RTL 到 KV260 板级运行的验证和性能诊断体系。本文使用 Icarus Verilog、Vivado XSIM、外部 RTL golden、链式 semantic golden 和 Vitis board smoke 构成多层验证流程；在硬件中加入 stage、sub-stage、raw-HWC、prefetch、collector、pass timeline 等计数器，用于定位性能瓶颈和解释优化收益。

1.4 论文组织结构

本文共分为六章。第一章介绍研究背景、相关研究现状和本文贡献。第二章给出卷积计算、INT8 量化、systolic array、YOLOv3-tiny 层结构和 MPSoC 平台基础。第三章提出 LASA 层自适应脉动阵列架构，介绍 PS/PL 职责划分、MPSoC 总体结构、LASA 数据通路、层自适应机制和链式推理执行流程。第四章讨论面向链式推理的数据流与调度优化，包括 KCS 三维分块执行模型、BSD 软硬件流式协同、OCRr 片上重排与重放、OPF-P 反馈重叠与预取。第五章介绍运行时集成、验证机制与性能诊断，包括 layer

descriptor、STREAM_CFG 配置、golden 验证路径、性能计数器和最终推荐软硬件组合。第六章给出正确性验证、板级端到端性能、消融实验、资源时序结果和后续工作讨论。

第二章 背景与相关工作

本章介绍本文设计所依赖的基础概念，包括卷积计算模型、INT8 量化、systolic array 数据流、YOLOv3-tiny 单尺度网络结构、MPSoC 平台特征以及 FPGA CNN 加速器相关研究。与传统背景介绍不同，本章重点服务于后文的数据流与调度优化：真实网络的性能瓶颈来自层形状、K pass 数量、输出通道分块、空间 tile、部分和反馈和 PS/PL 数据搬运的共同作用。

2.1 卷积计算与分块表示

给定输入特征图 $I \in \mathbb{R}^{H \times W \times C_{in}}$ 、权重 $W \in \mathbb{R}^{K_h \times K_w \times C_{in} \times C_{out}}$ 和偏置 B ，标准二维卷积可以表示为

$$O[y, x, c_o] = B[c_o] + \sum_{c_i=0}^{C_{in}-1} \sum_{k_y=0}^{K_h-1} \sum_{k_x=0}^{K_w-1} I[yS+k_y-P, xS+k_x-P, c_i] \cdot W[k_y, k_x, c_i, c_o], \quad (2.1)$$

其中 S 为 stride， P 为 padding。将卷积窗口和输入通道展开后，可得到 GEMM 形式：

$$O[p, c_o] = B[c_o] + \sum_{k=0}^{K_{total}-1} X[p, k] \cdot W[k, c_o], \quad K_{total} = C_{in} K_h K_w. \quad (2.2)$$

本文硬件按照三个维度组织计算： K 维分块、输出通道分块和空间分块。若 systolic array 每次处理 K_{tile} 个展开通道，则一个卷积层需要

$$N_{kpass} = \left\lceil \frac{K_{total}}{K_{tile}} \right\rceil \quad (2.3)$$

轮 K pass。若每个输出通道块包含 C_{out_tile} 个通道，则输出通道块数为

$$N_{cout} = \left\lceil \frac{C_{out}}{C_{out_tile}} \right\rceil. \quad (2.4)$$

真实层的调度规模由 $N_{tile} \times N_{cout} \times N_{kpass}$ 决定。后端层如 $13 \times 13 \times 512 \rightarrow 13 \times 13 \times 1024$ 虽然空间尺寸较小，但 K_{total} 和 N_{cout} 很大，因此容易暴露 pass 边界和 partial-PSUM 反馈瓶颈。

2.2 INT8 量化与后处理

边缘 CNN 推理通常采用 INT8 量化降低存储带宽和乘加资源消耗。浮点值 r 与量化整数 q 之间可表示为^[13]

$$q = \text{clamp} \left(\text{round} \left(\frac{r}{S} \right) + Z, q_{\min}, q_{\max} \right), \quad (2.5)$$

其中 S 为 scale, Z 为 zero point。卷积内部使用 INT8 输入和权重, 乘加结果累加到 INT32 partial sum。输出时通过逐通道 multiplier、shift 和 output zero point 完成重量化:

$$q_{\text{out}} = \text{clamp} \left(((p_{\text{sum}} \cdot m) \gg s) + Z_{\text{out}}, 0, 255 \right). \quad (2.6)$$

本文加速器将重量化、activation LUT 和可选 maxpool 集成在 PL 端, 从而减少每层输出回到软件后再处理的开销。A53 软件只负责写入每个输出通道 lane 的量化参数和 activation LUT 字节。

2.3 Systolic Array 与数据流

Systolic array 由规则排列的处理单元 (Processing Element, PE) 组成, 数据在相邻 PE 之间按节拍传播, 适合矩阵乘和卷积等规则计算^[5]。不同驻留策略会形成不同数据流: weight-stationary 将权重保持在 PE 内部, input-stationary 保持输入局部性, output-stationary 保持 partial sum, row-stationary 则试图综合利用多类数据复用^[7,9]。

本文采用 weight-stationary 风格。每个 K pass 开始前, 软件或 batch stream 将当前 $K_{\text{tile}} \times C_{\text{out_tile}}$ 权重块送入片上权重存储; 随后 IFM/window 向量流入阵列, PE 执行 INT8 乘法并累加到 INT32 partial sum。该选择的优点是权重加载和阵列计算语义清晰, 便于在 RTL 中实现稳定的 AXI-Stream 边界; 代价是多 K pass 层需要显式 partial-PSUM 反馈路径。

2.4 YOLOv3-tiny 单尺度网络

本文面向的模型是 YOLOv3-tiny 的单尺度链路。输入为 $416 \times 416 \times 3$ 量化 RGB 图像, 硬件链式执行 Conv0 到 Conv9, 最终输出 $13 \times 13 \times 24$ 检测头。表 2.1 给出本文 runtime 使用的层级形状和调度规模。

图 2.1 展示 YOLOv3-tiny 的单尺度检测链路。本文实现和验证的 Conv0–Conv9 对应该链路中从输入到 13×13 检测头的顺序卷积路径, 保留 3×3 卷积、 1×1 检测头、池化和逐层后处理等结构特征, 作为 LASA 层自适应机制与后续数据流优化的实验对象。

从表中可以看到, 后端 13×13 层虽然空间尺寸固定, 但 K passes 和 Cout blocks 显著增加, 是本文优化 partial-PSUM、raw-HWC replay 和 pass-level prefetch 的主要目标。

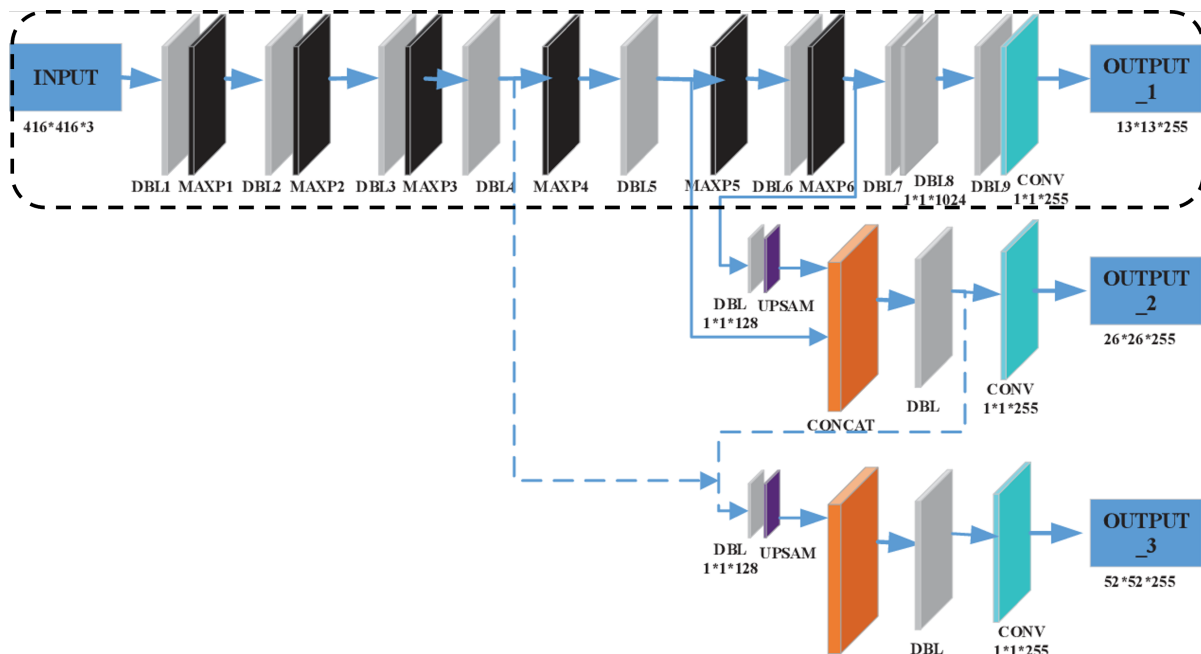


图 2.1 YOLOv3-tiny 单尺度检测链路

2.5 MPSoC 平台与 AXI 接口

Kria KV260 使用 Xilinx Zynq UltraScale+ MPSoC，包含 ARM Cortex-A53 PS 和 XCK26 PL 逻辑资源。PS/PL 之间通过 AXI4-Lite、AXI4-Stream 和 AXI4-MM 互连进行控制和数据搬运^[14-15]。本文采用 AXI-Lite 作为寄存器配置接口，并使用四路 AXI DMA：bias、weight、IFM 三路从 DDR 到 PL，OFM 一路从 PL 到 DDR。

AXI-Stream 的 TVALID/TREADY 握手机制有利于处理 PL 内部背压，但也要求 RTL 对 TLAST、TKEEP、packet count 和 DMA length 做严格约束。本文的 batch stream、raw-HWC cache 和 OFM debug stream 都建立在该边界之上。

表 2.1 Conv0–Conv9 单尺度链路层参数

层名	IFM	C_{in}	C_{out}	Kernel	K passes	Cout blocks
conv0_pool	416×416	3	16	3x3	2	1
conv1_pool	208×208	16	32	3x3	8	2
conv2_pool	104×104	32	64	3x3	16	4
conv3_pool	52×52	64	128	3x3	32	8
conv4_pool	26×26	128	256	3x3	64	16
conv5	13×13	256	512	3x3	128	32
conv6	13×13	512	1024	3x3	256	64
conv7	13×13	1024	256	1x1	57	16
conv8	13×13	256	512	3x3	128	32
conv9	13×13	512	24	1x1	29	2

2.6 相关工作比较

现有 FPGA CNN 加速器工作在自动化映射、阵列生成、模型压缩和单层吞吐上取得了大量成果^[3-4,10-12]。Chen 等从硬件资源利用与计算密度角度研究了 FPGA CNN 加速器设计^[16]，说明在受限可编程逻辑资源上需要同时考虑计算并行度与硬件开销。与这些工作相比，本文的差异主要在三个方面。

首先，本文以完整 Conv0–Conv9 链式推理作为验证对象，而不仅是单层或合成测试。每层输出都需要作为下一层输入，golden 数据也必须遵循同一条 RTL semantic 链路。其次，本文关注真实 MPSoC 部署中的数据搬运问题，包含 A53 runtime、AXI DMA、DDR 图像包、OFM packet 重排和 YOLO decode。最后，本文通过性能计数器对瓶颈迁移进行解释，强调优化为什么有效以及下一步瓶颈在哪里，而不是只报告单个最终延迟。

因此，本文的贡献更接近系统型和数据流优化型研究：在资源受限的可编程平台上，将真实 CNN 链路、RTL 数据通路、软件调度和板级性能诊断统一起来。

第三章 LASA 层自适应脉动阵列架构

本文的目标不是单独实现一个卷积算子，而是在边缘 MPSoC 上完成从 DDR 图像输入、PS 侧调度、PL 侧卷积计算、OFM 回写到 YOLO decode 的完整链式推理。为此，本文提出一种层自适应脉动阵列加速器（Layer-Adaptive Systolic Accelerator, LASA）。LASA 以统一 INT8 systolic array 为计算基础，通过 layer descriptor 和 AXI-Lite 配置支持不同卷积核、输出通道分块、K pass、量化参数、激活和池化组合，从而在同一套硬件上覆盖 YOLOv3-tiny Conv0–Conv9 的多种层形状。

3.1 系统设计目标与软硬件分工

真实 YOLOv3-tiny 链式推理对加速器提出三类要求。第一，PL 侧卷积核心需要保持稳定的 INT8/INT32 计算语义，使 Conv0–Conv9 的输出能够与 golden 逐层比较。第二，PS/PL 边界需要承载真实层的数据搬运和背压，而不是依赖小规模仿真中的理想输入。第三，系统必须暴露足够的计数器和调试接口，使端到端延迟可以被分解为 DMA、IFM 重排、阵列计算、partial-PSUM 反馈、OFM 后处理和软件 decode 等阶段。

因此，本文采用“PS 负责任务组织，PL 负责规则密集计算”的协同方式。PS 侧 A53 程序维护 layer descriptor、DDR 缓冲区、DMA 启停、输出重排和 YOLO decode；PL 侧 LASA 接收寄存器配置和 AXI-Stream 数据，在片上完成卷积、重量化、激活、池化和 OFM packet 输出。第四章讨论的 KCS 执行模型以及 BSD、OCRR、OPF-P 三类优化策略均建立在这一软硬件边界之上。

3.2 MPSoC 总体结构

图 3.1 给出系统级结构。DDR 中保存输入图像包、量化权重、中间特征图和输出 packet；PS A53 通过 AXI-Lite 配置 LASA，并通过四路 AXI DMA 搬运 bias、weight、IFM 和 OFM 数据。LASA 以 AXI-Stream 接收三路输入流，以 AXI-Stream 输出 OFM debug packet 到 DDR，随后由软件重排并执行 YOLO decode。

该结构分离控制面和数据面。AXI-Lite 只承载层参数、tile 参数、量化参数、stream 配置和性能计数器；大规模张量数据全部通过 AXI DMA 搬运。这一划分使 PL 内部数据流可以持续演进，而外部软件 ABI 和 DMA 拓扑保持稳定。

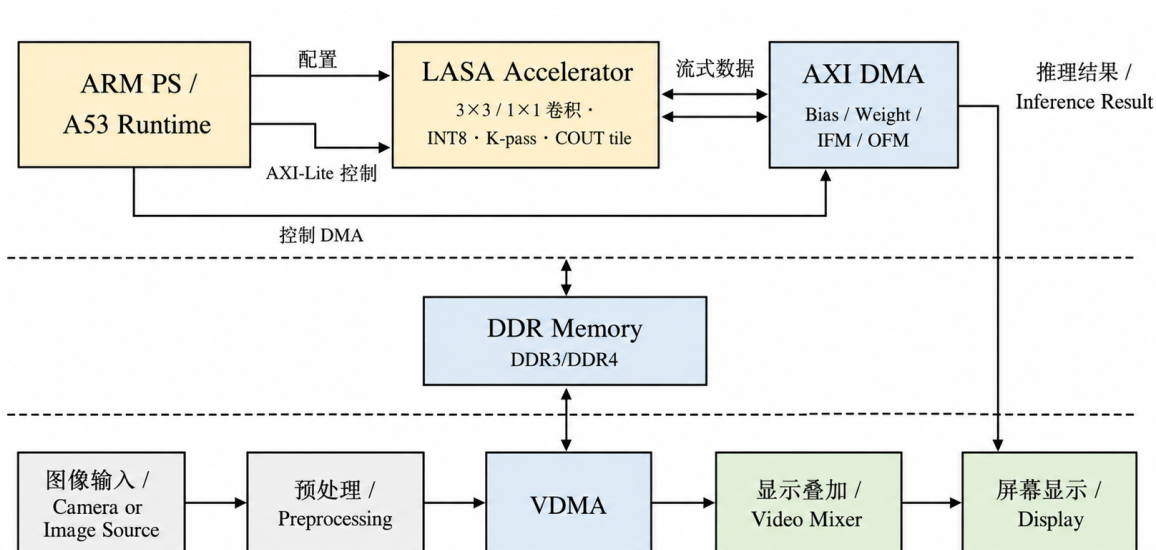


图 3.1 MPSoC 端到端系统结构

3.3 LASA 数据通路概览

图 3.2 展示 LASA 的数据通路。输入侧包含 bias loader、weight loader 和 IFM feeder；中间是 layer scheduler、systolic array 和 partial-PSUM 存储；输出侧包含 collector/drain、requant、activation、pooling 和 OFM writer。该图只描述各功能块的连接关系，具体的 KCS 分块、片上重排、PSUM 反馈和 pass 级预取在第四章展开。

LASA 的最终板级配置采用 $ROWS=18$ 、 $COLS=8$ 、 $Cout_{tile}=16$ 。阵列沿 K 维接收 18 路 IFM lane；每个列位置包含两个 INT8 output-channel lane，因此 $COLS=8$ 对应 16 个并行输出通道。两个 lane 的乘积分别累加到独立 INT32 partial sum 中，并在 final pass 后进入逐通道重量化路径。该 dual-lane INT8 column 设计提高了输出通道维度并行度，也决定了 PSUM packet、requant lane 和 OFM writer 的基本粒度。

3.4 层自适应机制

LASA 的“层自适应”体现在四个方面。第一，计算路径原生支持 3×3 与 1×1 卷积。对 3×3 卷积，IFM feeder 根据 oy, ox, k_y, k_x, c_i 生成展开后的 K 维向量；对 1×1 卷积，硬件使用 native vector path，避免将 1×1 强行映射为稀疏 3×3 后产生无效 K pass。这样 Conv7 和 Conv9 等检测头可以使用更短的 K_{total} 。

第二，后处理路径采用可配置旁路。final K pass 输出首先进入逐通道重量化模块；重量化后的 uint8 输出进入 activation LUT，可根据层描述选择 identity、ReLU/Leaky ReLU 等语义。若当前层启用 pooling，则 OFM pooling 模块执行 2×2 、stride=2 的 maxpool；若不启用 pooling，则数据直接旁路到 OFM byte stream。该机制使同一套硬件可以覆盖

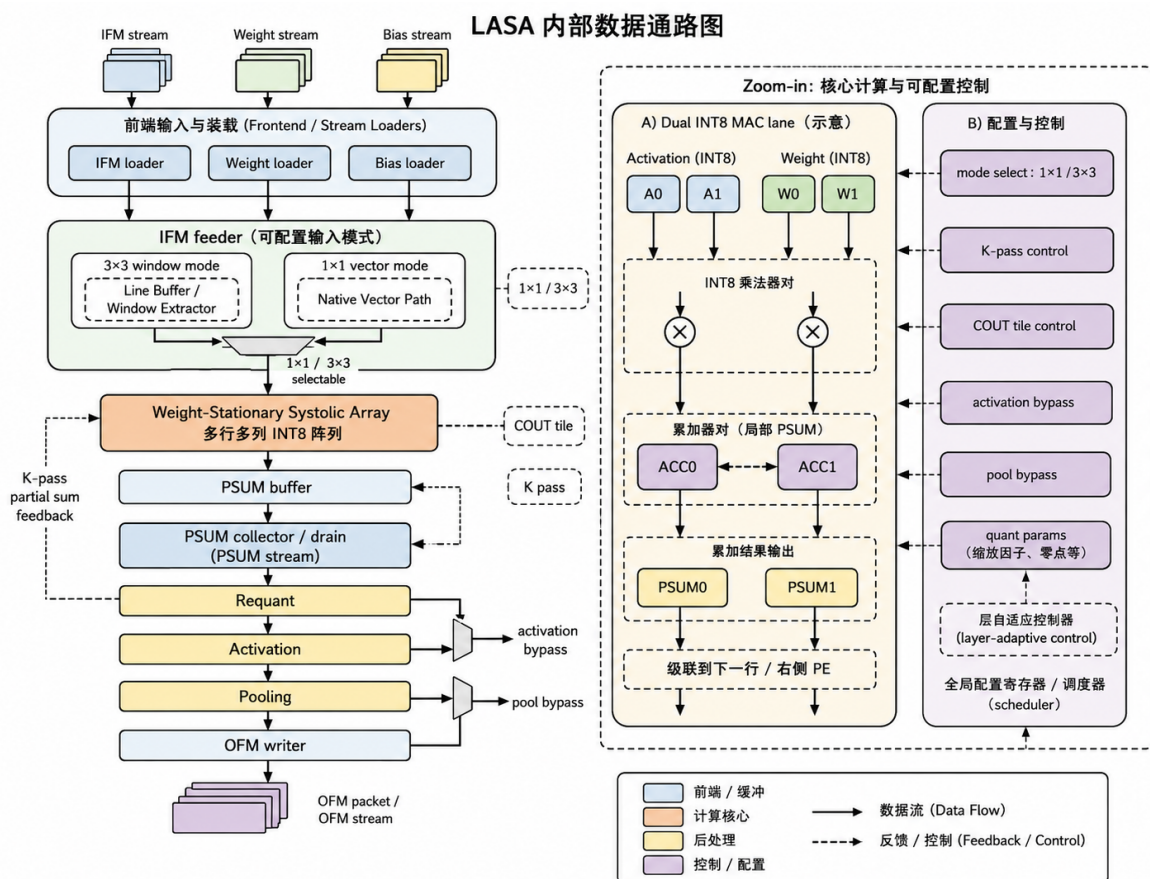


图 3.2 LASA 数据通路概览

YOLOv3-tiny 中带 pool 和不带 pool 的层。

第三，层参数由 descriptor 驱动。软件通过 layer descriptor 配置输入/输出尺寸、kernel、stride、padding、 K_{total} 、Cout blocks、spatial tile、量化参数、activation 和 pool 标志。硬件不需要为不同层重新综合，而是通过 AXI-Lite 寄存器和 stream packet 边界切换执行语义。

第四，数据流模式可组合。LASA 的基础数据通路可以与 batch stream、raw-HWC cache、partial-PSUM overlap、continuous collector 和 during-compute prefetch 等模式组合使用。对于以 $1 \times 1/3 \times 3$ 标准卷积、逐层激活和池化为主的轻量化边缘 CNN，该机制具备扩展到其他顺序型卷积层的基础；对 depthwise/group convolution、residual add、concat 等结构，则需要后续扩展专门的数据路径或软件融合策略。

3.5 链式推理执行流程

一次 Conv0–Conv9 推理从 DDR 中的图像包开始。软件读取或准备输入图像，按 layer descriptor 写入当前层的尺寸、量化、tile 和 stream 配置，随后启动 OFM S2MM

DMA 以及 bias、weight、IFM 三路 MM2S DMA。LASA 在当前层结束后输出带地址的 OFM packet，软件将其重排为 HWC buffer，并作为下一层输入或最终 YOLO decode 输入。

当前 OFM 输出采用 debug packet 语义：每个有效输出 byte 携带地址并写入 DDR。该格式不是最高带宽效率的连续 HWC burst，但保留了地址信息，便于板级阶段进行逐层重排、golden 比对和错误定位。第五章将进一步说明 runtime、寄存器接口、验证模式和性能计数器如何围绕该路径组织。

3.6 本章小结

本章提出并定义了 LASA 层自适应脉动阵列架构。LASA 通过 dual-lane INT8 systolic array、1x1/3x3 原生计算路径、可旁路后处理和 descriptor-driven 配置，为 YOLOv3-tiny 链式推理提供统一硬件执行基础。下一章将在 LASA 上进一步提出 KCS 三维分块执行模型，并围绕 PS/PL 交互、IFM 数据排布和 K-pass 边界气泡展开优化。

第四章 面向链式推理的数据流与调度优化

LASA 提供了层级可配置的硬件执行基础，但真实 YOLOv3-tiny 链式推理的端到端延迟仍不能只由阵列峰值吞吐决定。权重加载、IFM 打包、DMA 启动、tile 边界同步、partial-PSUM 反馈、pass 准备和 OFM 输出都会形成非计算开销。因此，本文将优化目标写为

$$T_{noncomp} = T_{layer} - T_{compute_fire}. \quad (4.1)$$

其中 $T_{compute_fire}$ 表示阵列真正接受输出 pixel 计算的周期数，其余部分均属于等待、搬运、反馈或边界开销。

本章首先提出 KCS 三维分块执行模型，将不同层统一映射为 K-pass、Cout block 和 spatial tile。随后围绕三类非计算瓶颈提出三类优化策略：面向 PS/PL 交互的批量流式协同策略 BSD (Batched Streaming Dataflow)、面向 IFM 布局转换的片上重排与重放策略 OCRR (On-Chip Reorder and Replay)，以及面向 K-pass 边界气泡的反馈重叠与预取策略 OPF-P (Overlapped PSUM Feedback and Prefetch)。

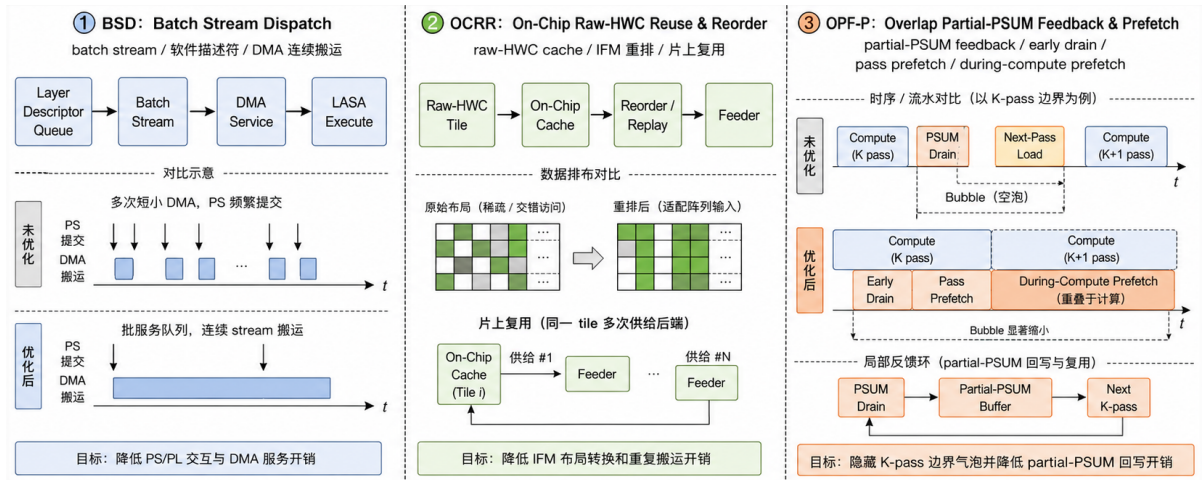


图 4.1 BSD、OCRR 与 OPF-P 三类优化策略总览

4.1 KCS 三维分块执行模型

KCS 执行模型由 K 维、输出通道和空间三个调度维度组成。K 维分块由 ROWS 决定，最终配置 ROWS=18，因此 $K_{tile} = 18$ 。输出通道分块由 COLS 和 dual-lane INT8 column 决定，每列产生两个输出通道 lane，因此

$$Cout_{tile} = 2 \times COLS. \quad (4.2)$$

最终配置 COLS=8, 对应 $Cout_{tile} = 16$ 。空间分块由 tile 的输出行范围决定, 用于限制片上 IFM/OFM buffer 和单次 AXI DMA 长度。

图 4.2 给出三维调度关系。每个 spatial tile 内, 调度器先遍历输出通道块, 再对当前输出通道块执行所有 K pass。首个 K pass 从 bias 初始化 partial sum; 非首 pass 读取上一轮 partial-PSUM; final pass 将结果送入 requant、activation、pooling 和 OFM path。

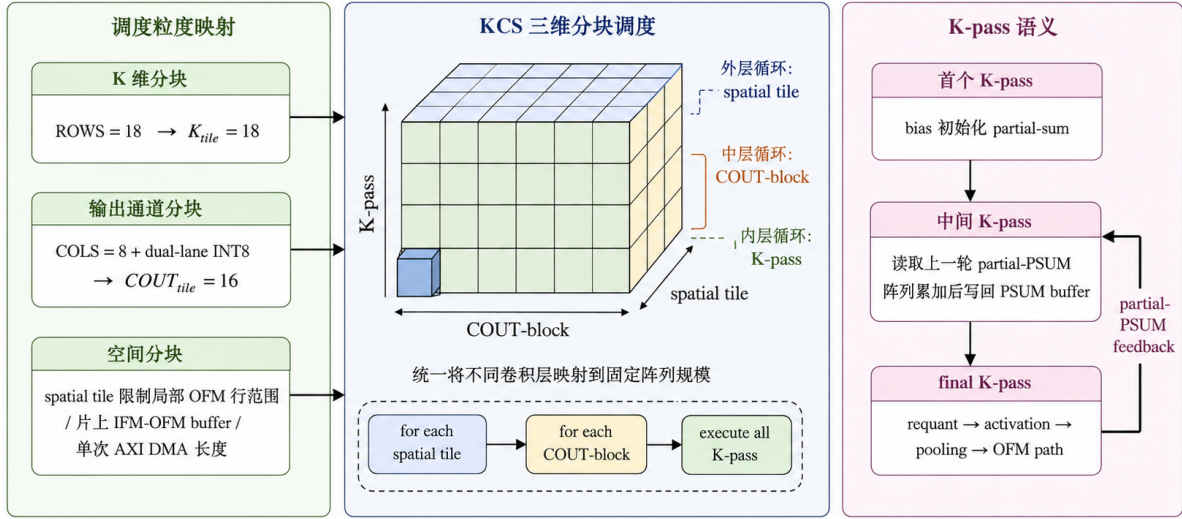


图 4.2 KCS 三维分块执行模型

KCS 不是单独的优化策略, 而是 LASA 上的统一映射抽象。它回答不同 CNN 层如何映射到固定阵列规模, 而 BSD、OCRR 和 OPF-P 则分别针对映射后暴露出的 PS/PL 交互、数据排布和 pass 边界开销进行优化。

4.2 BSD: 面向 PS/PL 交互的批量流式协同

BSD 的目标是降低 PS 逐请求服务和 DMA 频繁启动造成的系统协同开销。早期 per-request 模式中, 硬件每需要一个 bias block、weight tile 或 IFM line 时向软件发出请求, 软件随后打包数据、启动 DMA 并等待完成。这种方式适合小型 smoke test, 但在 Conv6 等真实层中会产生大量服务事件, A53 侧开销和 DMA 启动开销会淹没阵列计算。

BSD 将每个 spatial tile 所需的 bias、weight 和 IFM 数据预先打包成连续 AXI stream。软件在 tile 开始时分别启动三路输入 DMA, 硬件通过 packet count 恢复内部边界, 只在整个 stream 末尾依赖 TLAST。这样, DMA 启动次数从每个请求一次降低为每个 tile 每类输入一次, AXI backpressure 也由 PL 自然吸收。

权重数据在模型中通常按输出通道、输入通道和 kernel 维度组织, 而硬件消费顺序是 $Coutblock \rightarrow Kpass \rightarrow lane \rightarrow channel$ 。本文在生成 Vitis header 时直接产生硬件

消费顺序的权重数组，使 weight DMA 可以读取 ELF 只读段中的连续数组，不再需要运行时 scratch copy 或 repack。PL 端 weight loader 使用 64-bit AXI beat 写入多个 byte bank，进一步降低 weight load 等待。预打包权重虽然作用于数据布局，但其主要收益是减少运行时 PS 服务和 DMA 准备成本，因此归入 BSD。

4.3 OCRR: 面向 IFM 布局转换的片上重排与重放

OCRR 的目标是缓解软件自然 HWC 布局与 systolic array K-lane 消费顺序之间的不匹配。IFM 输入存在两种路径。第一种是 prepacked IFM stream: 软件提前把每个 K pass 需要的 IFM lane 按硬件顺序打包。该方式 PL 简单，但 A53 侧打包成本较高。第二种是 raw-HWC stream: 软件发送自然 HWC tile，PL 在片上缓存后按 pass 和窗口顺序 replay。

图 4.1 中间部分给出 OCRR 的数据排布思路。prepacked 路径把重排压力放在软件；raw-HWC 路径把重排压力转移到 PL cache/replay，但可减少 A53 逐 pass 复制和索引开销。对 3x3 层，cache materializes 每个输出 pixel 的 3x3 窗口，并按 18 lane 输出给阵列；对 1x1 层，cache 按连续通道向量 replay。

OCRR 还包括后端 full-tile 调度。Conv5、Conv6 和 Conv8 的空间尺寸均为 13×13 。早期调度将这些层拆成 $4 + 4 + 4 + 1$ 行 spatial tile，可以降低片上缓存容量需求，但每个 tile 都需要重复启动 bias、weight、IFM 和 OFM 流程。随着 raw-HWC cache 使用 URAM 扩容，后端层可以使用完整 13×13 tile，从而减少 tile 边界固定成本。容量条件可近似写为

$$tile_pixels \times \left\lceil \frac{C_{in}}{2} \right\rceil \leq HWC_CACHE_DEPTH. \quad (4.3)$$

在最终 full-tile build 中，Conv5/8 需要 $13 \times 13 \times \lceil 256/2 \rceil = 21632$ 个 materialized word，Conv6 需要 $13 \times 13 \times \lceil 512/2 \rceil = 43264$ 个 materialized word。raw-HWC 并非对所有层都收益为正；Conv3 raw-HWC 虽然可功能通过，但由于 tile/FIFO 容量和 replay 开销，板级结果不优于推荐配置，因此作为边界分析而非默认路径。

4.4 OPF-P: 面向 K-pass 边界的反馈重叠与预取

OPF-P 的目标是隐藏多 K pass 之间的 partial-PSUM 反馈和下一 pass 准备气泡。对于 $K_{total} > K_{tile}$ 的层，第一轮 K pass 只能产生部分和，后续 pass 必须读取前一轮 partial sum 并继续累加，最后一轮才进入重量化和输出路径。对一个输出 pixel 和一个 $Cout_{tile}$ 通道块，硬件内部形成

$$P[p, c] = \sum_{k=k_0}^{k_0+K_{tile}-1} X[p, k] \cdot W[k, c]. \quad (4.4)$$

当当前 pass 不是 final pass 时，该 packet 被写入片上 partial-PSUM buffer；下一 pass 读取同一地址的 partial sum 作为初值继续累加。当当前 pass 是 final pass 时，该 packet 被送入 requant/activation/pool/OFM path。

该语义要求 partial-PSUM 地址、Cout block 和 pass context 与下一 pass 严格对齐，同时避免读写冲突，并保持 final pass 输出顺序与软件重排和 golden 比对一致。本文使用 ping-pong bank 和 per-pass context 维护这些条件。也就是说，PSUM 反馈不是独立于数据流之外的模块，而是跨 K pass 数据流的一部分。

若每轮 pass 都严格串行执行“计算完成、全部 drain、下一轮 feed、下一轮 compute”，后端大层会出现明显阵列空泡。本文首先将 drain writer 改为 AXI-Stream 风格握手：packet valid、addr 和 data 在真实 *valid&ready* 握手前保持稳定，握手成功后才推进地址。随后加入 read-return tracker 和 one-entry hold register，使下游 ready 时可以接近每周输出一个 PSUM packet。

Early drain 允许调度器在当前 pass 已经开始产生 PSUM 数据后提前启动 drain，只要 partial-PSUM FIFO 已具备可读数据。Continuous PSUM collector 进一步改变非 final pass 的路径：collector 在 compute 开始前接收 pass context，并在阵列 FIFO 产生完整 packet 后立即将其写入 partial-PSUM bank；final pass packet 则进入后处理流水线。这样，非 final pass 的 partial-PSUM 写回从“compute 后集中 drain”变为“compute 过程中持续收集”，减少 pass 边界处的反馈延迟。

K pass 之间除了 PSUM 反馈，还包含下一 pass 权重加载和 IFM replay。Pass-level prefetch 在当前 pass 已经满足安全条件后提前准备下一 pass 的 weight 和 raw-HWC replay。该机制遵循两个约束：一是不能覆盖当前 pass 正在使用的 PE 权重；二是 next-pass IFM replay 只能进入解耦 FIFO 或安全队列，不能破坏当前 pass 的输入顺序。During-compute prefetch 进一步允许在当前 pass compute 进行时启动下一 pass 权重和 raw-HWC replay 的 staging，从而降低 compute stage 中由于下一 pass 准备不足造成的 idle。

图 4.3 对比了串行 pass 和 OPF-P 重叠 pass 的时间线。

4.5 优化策略小结

表 4.1 汇总了 KCS 执行模型与三类优化策略的定位。KCS 负责将不同卷积层统一映射到 LASA；BSD 作用于软硬件协同层，减少 PS/DMA 服务开销；OCRR 作用于数据排布层，缓解 IFM 布局转换和 tile 边界开销；OPF-P 作用于流水线调度层，隐藏 K-pass 反馈和下一 pass 准备气泡。

三类优化共同目标是减少 $T_{noncomp}$ ，但它们针对的不是同一种开销。这样的分层表述使本文的优化链条从零散实现点收敛为系统协同、数据排布和流水线调度三个层面

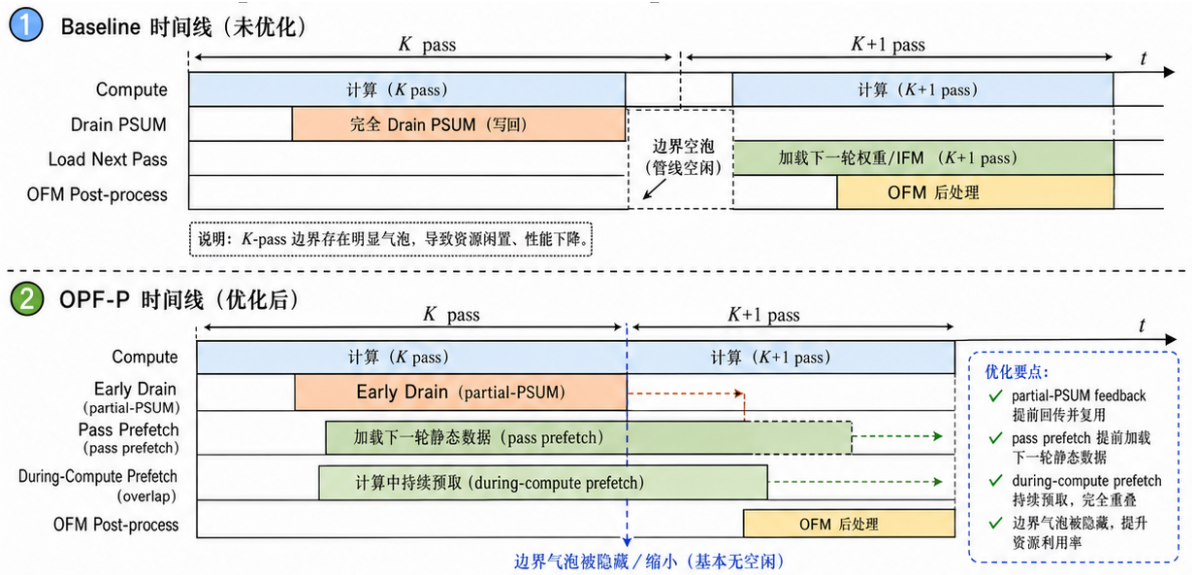


图 4.3 OPF-P 反馈重叠与 pass 级预取时间线

表 4.1 KCS 执行模型与三类优化策略

名称	层次	主要瓶颈	对应机制
KCS	执行模型	层形状到固定阵列的映射	K-pass、Cout block、spatial tile
BSD	软硬件协同	PS 服务与 DMA 启动	batch stream、packet count、prepacked weight
OCRR	数据排布转换	IFM 重排与 tile 边界	raw-HWC cache、replay、backend full-tile
OPF-P	流水线调度	PSUM/pass 边界气泡	drainpipe、continuous collector、pass prefetch

的策略。下一章将说明这些策略如何通过 runtime、寄存器接口、验证流程和性能计数器集成为可运行、可诊断的板级系统。

第五章 运行时集成、验证机制与性能诊断

第四章提出了 KCS 执行模型以及 BSD、OCRR、OPF-P 三类优化策略。本章转向系统实现层面，说明软件 runtime 如何配置 LASA 与这些策略，板级验证如何保证 Conv0–Conv9 链式推理正确，以及性能计数器如何支撑第六章的消融分析。通过运行时、寄存器接口、验证流程和计数器体系，本文将前述架构和优化策略落实为可运行、可复现、可诊断的板级系统。

5.1 Layer Descriptor 与链式调度

Vitis bare-metal runtime 使用静态 layer descriptor 描述 Conv0–Conv9 的输入尺寸、输出通道数、kernel、stride、pool、K passes、Cout blocks、tile 形状、量化参数和 golden 信息。这些字段对应 LASA 的层自适应配置和 KCS 执行模型。运行时按 layer 和 spatial tile 循环，写入配置寄存器，启动 OFM S2MM DMA，再启动 bias、weight、IFM 三路 MM2S DMA，最后等待 accelerator done 和 OFM DMA 完成。

对前端较大空间尺寸层，runtime 根据 tile 行范围逐块搬运输入和输出；对后端 13×13 层，最终推荐配置启用 backend full-tile，减少 tile 边界启动次数。每层完成后，软件将 OFM packet 重排为 HWC buffer，并作为下一层输入。Conv9 输出进入 YOLO decode，形成 DDR image demo 的最终检测结果。

5.2 AXI-Lite 配置接口

AXI-Lite 是 PS runtime 配置 PL 端 LASA 的控制面。为避免混淆，本文将 AXI-Lite 地址空间分为两类：第一类是**运行配置寄存器**，由软件在每个 layer 或 spatial tile 启动前写入，直接决定本次硬件执行语义；第二类是**只读性能计数器**，由硬件在运行过程中累加，软件在层结束后读取，用于第六章的性能分析。二者共享同一个 AXI-Lite 端口，但作用不同：前者参与功能执行，后者只参与观测和诊断。

表 5.1 给出本文最终系统中实际用于工作的主要配置寄存器。完整地址空间还包含若干调试状态位和实验性 trace 选择位，论文中只列出与 LASA 层自适应、KCS 映射以及 BSD/OCRR/OPF-P 优化直接相关的配置项。

所有主要优化均通过 STREAM_CFG 位控制。bit0 使能 BSD 的 batch mode，bit1 使能 OCRR 的 raw-HWC mode，bit2 使能 OPF-P 中的 early drain，bit3 使能 pass prefetch，bit4 使能 partial-PSUM overlap，bit5 使能 continuous PSUM collector，bit7 使能 during-

表 5.1 主要运行配置寄存器

类别	代表寄存器	作用
控制状态	CTRL/STATUS	写 <code>start</code> 启动 accelerator, 写 <code>clear</code> 清除状态; 读 <code>busy</code> , <code>done</code> 和 <code>config_error</code> 判断本次 tile 是否完成或配置非法。
层形状配置	FM_SIZE, OFM_SIZE CONV, K_TOTAL COUT_TOTAL	配置输入/输出尺寸、stride、padding、native 1x1/3x3 模式、K 维总长度和输出通道数, 是 LASA 适配不同卷积层的基础。
tile 配置	NUM_PIXELS TILE_ROWS PIXEL_BASE EXPECTED_BYTES	配置当前 spatial tile 的输出 pixel 数、输出行范围、全局 HWC 地址基址和 OFM 期望输出字节数, 对应 KCS 中的 spatial tile 维度。
量化与后处理	IFM_ZP, POOL_CFG QUANT_ADDR/DATA LUT_ADDR/DATA	配置输入零点、pool bypass/maxpool、逐 lane 重量化参数和 activation LUT, 使同一硬件支持不同层的量化、激活和池化组合。
stream 与优化开关	STREAM_CFG BIAS/WEIGHT/IFM_PACKETS TAIL_CONFIG	配置 batch stream 的 packet 数、raw-HWC 路径、early drain、pass prefetch、partial-PSUM overlap、continuous collector 和 during-compute prefetch 等数据流与调度模式。

compute prefetch。bit6 对应 column-level PSUM, 是当前探索性设计, 不作为最终推荐配置的必要条件。

一次 layer/tile 的配置顺序如下。首先, runtime 确认 accelerator 处于 idle 状态, 并写入层形状、tile 范围、输入零点、池化配置和 OFM 期望输出字节数。随后, 软件写入当前输出 lane 的重量化参数, 必要时写入 activation LUT。若启用 batch stream 或 raw-HWC 路径, runtime 进一步写入 STREAM_CFG、三路输入 packet 数和 TAIL_CONFIG。最后, 软件先启动 OFM S2MM DMA, 再启动 bias、weight、IFM 三路 MM2S DMA, 并向 CTRL/STATUS 写 `start`。硬件运行完成后置位 `done`, runtime 再解析 OFM packet、读取性能计数器并进入下一层或下一 tile。

这种设计保持了软件 ABI 的稳定性。默认关闭实验开关时, 系统回到保守串行语义; 逐步打开开关时, 软件只需写入不同 STREAM_CFG 和少量 tail/prefetch 参数, 外部 AXI packet 格式保持不变。因此, 同一 runtime 可以执行 baseline、batch、raw-HWC、continuous PSUM 和 during-compute prefetch 等多个 A/B 配置。

5.3 正确性验证路径

本文采用从模块到板级的多层验证。RTL 层面使用 Icarus Verilog 和 Vivado XSIM 覆盖 PE、systolic array、feeder、AXI loader、PSUM buffer、collector、AXI-Lite bridge 和端到端 conv top。模块级测试重点检查握手稳定性、K pass 地址推进、partial-PSUM bank 切换、raw-HWC replay 顺序和 OFM packet 地址。

系统层面使用外部 golden 数据验证真实层，包括 Conv0、Layer06/Conv3、Conv4 以及后端 Conv5/6/8 raw-HWC tile。板级层面包含两条路径。第一条是 batch-chain 模式，对 Conv0–Conv9 每层输出与 RTL semantic golden 做 bit-exact 比较；第二条是 DDR image demo 模式，将 $416 \times 416 \times 3$ 图像包写入 DDR，由硬件执行十层推理并在 A53 侧进行 YOLO decode。

OFM debug packet 在验证阶段承担关键作用。每个输出 byte 携带地址，软件可以将其重排为 HWC buffer 并定位错误位置。虽然这种格式牺牲了带宽效率，但它使模块仿真、板级 UART 日志、golden 比对和最终 DDR demo 能够使用同一套语义闭环。

5.4 性能计数器设计

仅报告端到端延迟无法说明优化是否真正降低了目标瓶颈。为此，本文在 PL 端加入多层只读性能计数器，类别如表 5.2 所示。这些寄存器不改变硬件执行语义，只在 accelerator 运行期间记录事件和周期数；runtime 在每个 tile 或 layer 结束后读取并打印 UART 日志。

这些计数器使本文能够说明每次优化后的瓶颈迁移。例如，BSD 主要影响 DMA 启动与外部等待，OCRr 主要影响 raw-HWC load/replay 与 spatial tile 固定成本，OPF-P 主要影响 drain、collector、prefetch 和 compute idle。第六章的阶段 breakdown 和消融表均来自这些只读计数器或与其一致的 UART 日志；它们与表 5.1 中的运行配置寄存器分离，因此不会改变实验版本的功能语义。

5.5 最终推荐配置

为了避免多个实验版本并列造成主线混乱，本文将 D:/MPSoC/b_kprefetch_22 作为最终推荐硬件构建。该版本在 Vivado/Vitis 2022.2、100 MHz 目标频率下完成实现，采用 ROWS=18、COLS=8、 $C_{out_tile} = 16$ 的 LASA 阵列配置，并启用 BSD、OCRr 与 OPF-P 的主要机制，包括后端 raw-HWC full-tile、continuous PSUM collector 和 during-compute prefetch。

最终推荐软件配置为 conv0_conv9_ddr_demo，开启 RawHwcConv4、5、6、8，early

表 5.2 主要性能计数器类别

类别	代表寄存器	诊断对象
PERF	PERF_BUSY	顶层 busy 周期、外部等待周期和 compute fire 周期，用于区分计算时间与非计算开销。 layer scheduler 各阶段耗时，用于观察优化后瓶颈在 bias、weight、feeder、compute、drain 和 OFM post 之间的迁移。
STAGEPERF	PERF_WAIT_*	
	PERF_COMPUTE	
SUBPERF/ RAW-STAT	STAGE_*	细分 feeder push/FIFO stall、compute active/IFM stall、raw-HWC load/unpack/replay 等事件，用于分析 OCRR 和 compute idle。
	FEED_*	
	COMP_*	
DRAINPERF	RAW_*	统计 PSUM drain 读出、packet fire 和下游 back-pressure，用于评估 drainpipe 与 early drain。
PREFETCHPERF	DRAIN_*	统计下一 K pass 预取是否命中以及等待时间，用于评估 pass-level prefetch 和 during-compute prefetch。
COLLECTPERF	PREFETCH_*	统计 continuous PSUM collector 的 partial/final 写入和 context stall，用于评估 OPF-P 中的持续收集路径。
PASSPERF	COLLECT_*	统计 pass 粒度 first fire、fire span、selected pass timestamp 和 compute idle，用于定位 K-pass 边界气泡与阵列利用率。
	PASS_*	
	PASSTRACE_*	

drain、pass prefetch、partial-PSUM overlap、continuous PSUM、backend full-tile 和 during-compute prefetch。该组合既通过 Conv0–Conv9 batch-chain bit-exact 验证，也在两张 DDR image demo 上得到稳定 YOLO decode 结果，是第六章实验的主线配置。

5.6 探索性 Column-Level PSUM

当前 continuous PSUM collector 仍以完整 packet 为反馈粒度：collector 等待所有列 FIFO 都有数据后写入一个 $COLS \times 2$ 通道 packet。Column-level PSUM 的探索方向是按列独立写入和读取 partial sum，使下一 pass 能够按照 systolic array 的列传播延迟更早获得可用数据。

本文已经实现了 column-level ping-pong buffer 和 column stream feeder 的基础模块，并完成模块级验证。但该路径尚未作为最终推荐板级配置，因此本文将其作为未来工作讨论，不将其计入核心性能贡献。这样可以避免把尚未完整端到端验证的实验性设计包装为成熟成果。

5.7 本章小结

本章说明了 LASA 以及 KCS/BSD/OCRR/OPF-P 如何通过 Vitis runtime、AXI-Lite 寄存器、STREAM_CFG 开关、golden 验证路径和性能计数器形成完整板级闭环。这些机制使第六章能够同时报告正确性、端到端性能、阶段 breakdown、资源时序和后续工作分析。

第六章 验证与实验结果

本章报告 LASA 加速器的正确性验证、板级端到端性能、优化消融、阶段计数器分析和资源时序结果。实验围绕 KCS 执行模型以及 BSD、OCRR、OPF-P 三类优化策略展开，验证其在真实 MPSoC 环境中降低 YOLOv3-tiny 链式推理非计算开销的效果，并解释性能瓶颈如何迁移。

6.1 实验设置

实验平台为 Xilinx Kria KV260, PL 器件为 XCK26, 工具链为 Vivado/Vitis 2022.2。最终推荐硬件构建目录为 D:/MPSoC/b_kprefetch_22, 核心时钟目标为 100 MHz。LASA 参数为 ROWS=18、COLS=8、 $Cout_{tile} = 16$ 、IFM FIFO depth=1024, 后端 raw-HWC full-tile cache 使用 URAM 承载 Conv5/6/8 的 13×13 tile。

最终推荐软件配置为 conv0_conv9_ddr_demo, 开启 RawHwcConv4、5、6、8, early drain、pass prefetch、partial-PSUM overlap、continuous PSUM、backend full-tile 和 during-compute prefetch。验证路径包含两类：第一类是 batch-chain 模式，对 Conv0–Conv9 每层输出与 RTL semantic golden 做 bit-exact 比较；第二类是 DDR image demo, 将 $416 \times 416 \times 3$ 图像包写入 DDR，由硬件执行十层推理并在 A53 侧进行 YOLO decode。

6.2 正确性验证

本文采用从模块到板级的多层验证。RTL 层面使用 Icarus Verilog 和 Vivado XSIM 覆盖 PE、array、feeder、AXI loader、PSUM buffer、collector、AXI-Lite bridge 和端到端 conv top。系统层面使用外部 golden 数据验证真实 Conv0、Layer06/Conv3、Conv4 以及后端 Conv5/6/8 raw-HWC tile。板级层面使用 KV260 batch-chain 和 DDR image demo 验证完整链路。

需要强调的是，链式 golden 与单层 standalone golden 不可混用。由于前级硬件量化、激活和池化语义会影响后级输入，Conv5 之后的 expected output 必须由同一条 RTL semantic chain 生成。本文在 batch-chain 中逐层保存硬件输出并比较，从而验证层间数据重排、feature buffer 切换和检测头输出的一致性。

表 6.1 关键正确性验证结果

验证对象	覆盖内容	结果
Conv0 crop + pool	真实输入 crop、卷积、LUT、pool、OFM packet	0 mismatch
Layer06/conv3 pool	$52 \times 52 \times 64 \rightarrow 52 \times 52 \times 128$	XSIM 与板级通过
Conv4 pool	$26 \times 26 \times 128 \rightarrow 13 \times 13 \times 256$	XSIM 与板级通过
Conv5/6/8 raw-HWC Conv0–Conv9 batch-chain	后端 full-tile raw-HWC + overlap 配置 十层链式 RTL semantic golden	XSIM 2726/0 PASS
DDR image demo	动态图像输入与 YOLO decode	两张图像 PASS

6.3 端到端性能与优化消融

表 6.2 汇总了主要板级优化版本的端到端 DDR demo 延迟。所有结果均来自 KV260 板级运行，输入图像和模型链路保持一致。各版本展示了本文优化策略逐步降低非计算开销的过程。

表 6.2 端到端 DDR demo 优化消融

版本或优化阶段	主要变化	典型延迟
BSD baseline	batch stream 与 DDR 图像 demo 初版	≈ 1.18 s
BSD weight layout	预打包权重与 64-bit PL 解包	≈ 0.861 s
OPF-P drainpipe	流水化 PSUM drain writer	≈ 0.646 s
OPF-P prefetch	early drain 与下一 K pass 预取	≈ 0.387 s
OPF-P collector	非 final pass 持续写 partial-PSUM	≈ 0.371 s
OCRR full-tile	Conv5/6/8 使用 13×13 full tile	≈ 0.335 s
OPF-P during-compute	当前 compute 中准备下一 K pass	≈ 0.288 s

最终 b_kprefetch_22 构建完成 full-programming batch-chain 验证，日志中 UART detections 与 decode golden 匹配。两张 DDR image demo 的结果如表 6.3 所示。

表 6.3 最终 DDR image demo 结果

图像	总延迟	检测结果	日志
maksssksksss0.png	288.002 ms	with_mask, score=0.357321	20260616_193623
maksssksksss1.png	287.993 ms	with_mask, score=0.295050	20260616_193752

该结果约为 3.5 FPS。与早期约 1.18 s 的完整链式推理版本相比，最终版本将端到端延迟降低到约 0.29 s，获得约 4.1 倍端到端加速。该提升来自 BSD、OCRR 和 OPF-P 对 PS/PL 交互、IFM 布局转换和 K-pass 边界气泡的分别优化。

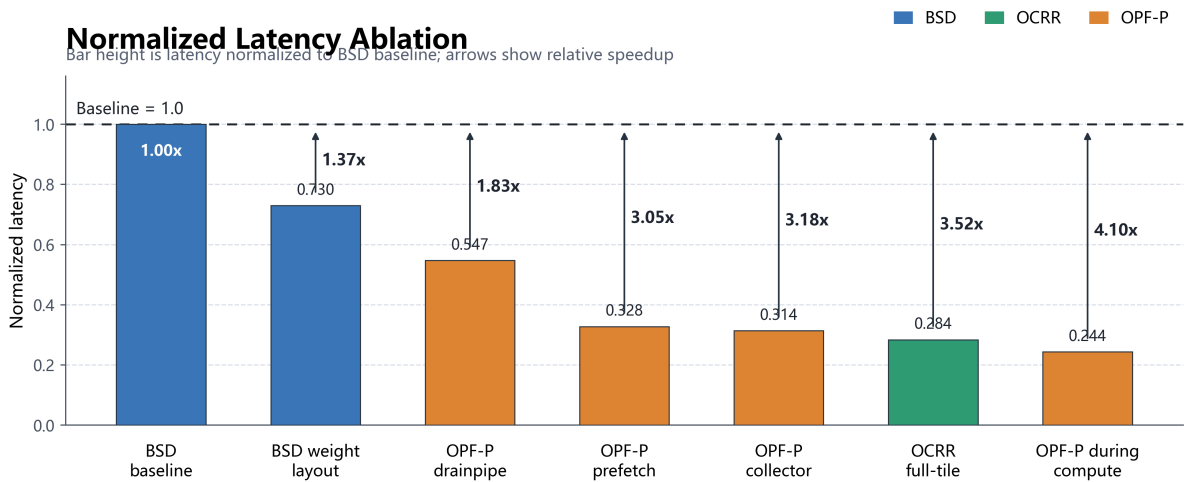


图 6.1 端到端 DDR demo 归一化延迟与相对加速比

为给出软件参考基线，本文采用单核 Cortex-A53 INT8 scalar KCO 实现作为 CPU-only baseline。该实现与硬件实验使用同一条 Conv0–Conv9 单尺度链路，板级日志记录的 CPU_TOTAL 为 2540175 us。图 6.2 将其与最终硬件 DDR image demo 的端到端延迟进行对比，硬件版本相对于该软件基线获得约 8.82 倍加速。

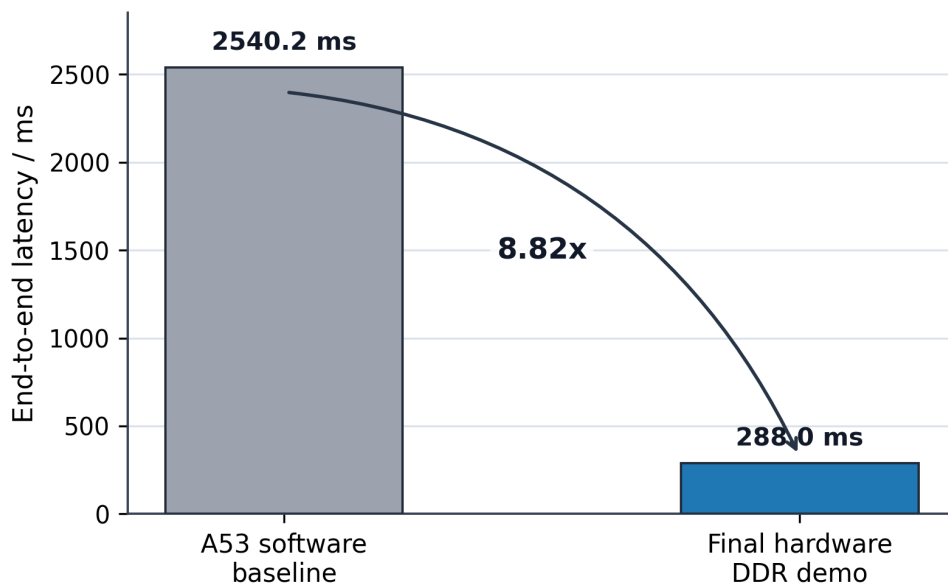
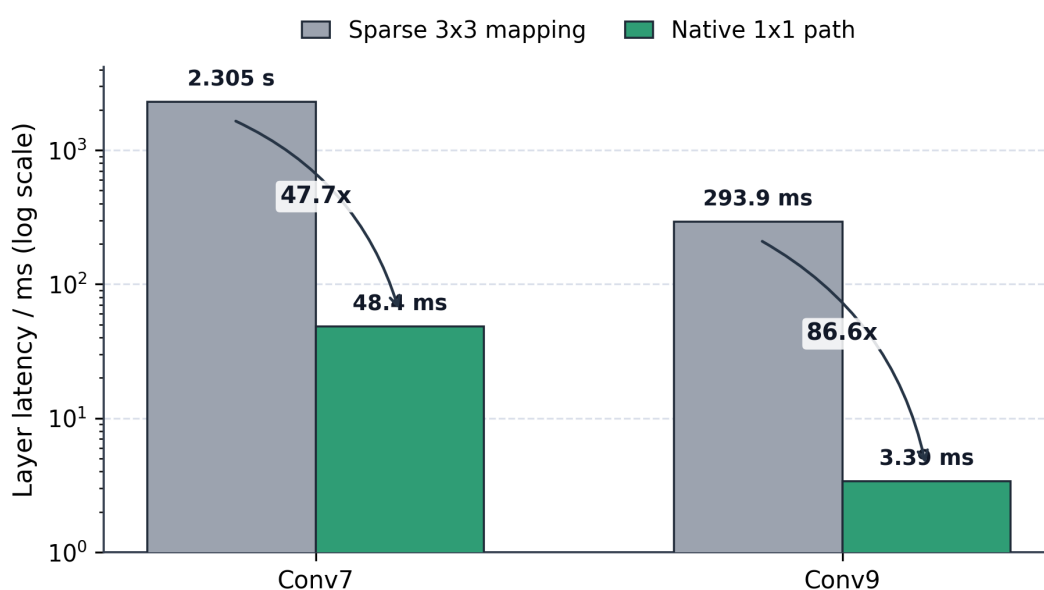


图 6.2 A53 软件基线与最终硬件端到端延迟对比

此外，LASA 在同一套阵列和输出后处理路径中原生支持 1×1 与 3×3 卷积。图 6.3 对比了早期将 1×1 卷积映射为稀疏 3×3 的实现，以及后续 native 1×1 路径在 Conv7 和 Conv9 上的单层延迟。原生路径避免了稀疏 3×3 映射引入的无效 K pass 与权重/输入搬运，Conv7 和 Conv9 分别获得约 47.7 倍和 86.6 倍加速。

图 6.3 1×1 卷积层原生路径与稀疏 3×3 映射延迟对比

6.4 阶段计数器分析

仅报告端到端延迟无法说明优化来源。表 6.4 给出最终 DDR demo 的关键硬件计数器换算时间。可以看到，最终硬件 busy 时间约为 247.184 ms，端到端 total 与 PL busy 之间的差额来自 A53 runtime、DMA 管理、cache maintenance、OFM 重排和 YOLO decode 等软件侧开销。

表 6.4 最终版本关键阶段计数器

指标	时间
Total DDR demo	288.002 ms
HW busy	247.184 ms
compute_fire	74.323 ms
feeder stage	99.913 ms
compute stage	121.022 ms
compute idle in stage	46.699 ms
drain stage	16.467 ms

这些计数器说明，经过 OPF-P 中的 drainpipe 和 continuous PSUM 后，drain 已不再是主要瓶颈；经过 OCRR full-tile 和 OPF-P during-compute prefetch 后，剩余开销主要分布在 feeder/replay 和 compute stage idle。换言之，下一步优化不应继续单纯扩大阵列，而应继续提高 raw-HWC replay 吞吐、减少 compute stage 内部空泡，并评估 OFM debug packet 对软件重排的影响。

6.5 资源与时序

表 6.5 给出最终推荐硬件构建的实现资源与时序。该版本在 Vivado 2022.2 下完成布局布线，WNS 为正，说明 100 MHz 目标可闭合。

表 6.5 最终推荐构建资源与时序

项目	数值
Build directory D:/MPSoC/b_kprefetch_22	
CLB LUTs	84480
CLB Registers	53260
BRAM Tile	63
URAM	24
DSP	184
WNS	+0.092 ns
TNS	0 ns
WHS	+0.010 ns
Routing errors	0

资源结果体现了本文优化的代价。OCRR raw-HWC full-tile cache 引入 URAM 以减少后端 tile 启动开销；OPF-P continuous PSUM collector 和性能计数器增加 LUT/FF；during-compute prefetch 增加调度与队列控制逻辑。整体上，设计仍能装入 XCK26 并闭合 100 MHz，但 LUT 使用已经较高，继续增加 column-level PSUM 或更深 overlap 时需要谨慎评估资源和时序裕量。

6.6 与相关工作的定性比较

表 6.6 从研究重点角度比较本文与代表性工作。由于不同论文使用的模型、输入尺寸、器件、频率、量化策略和是否计入软件后处理差异较大，本节采用网络粒度、数据流讨论、板级端到端验证和性能解释方式等维度进行定性比较。

本文的特点在于系统闭环和瓶颈解释。当前 runtime 面向 YOLOv3-tiny 单尺度静态调度，尚未扩展为通用 CNN 编译器；OFM 输出采用 debug packet 格式，便于验证和错误定位，后续可进一步迁移为连续 HWC burst 以提高带宽效率；功耗数据尚未形成板级实测，后续可结合 Vivado power estimate 或外部功耗计进行能效评估。

6.7 讨论与后续工作

基于当前实验结果，后续工作可从以下方向展开。第一，最终版本端到端延迟约为 288 ms，阶段计数器显示 feeder/replay 与 compute idle 仍占有较大比例，后续可继

表 6.6 与代表性工作的定性比较

特征	Angel-Eye ^[4]	S2A ^[10]	本文
主要目标	自动化映射流程	systolic array 生成	MPSoC 链式推理优化
网络粒度	多模型映射	阵列综合	YOLOv3-tiny Conv0–Conv9
数据流讨论	编译映射为主	阵列结构为主	BSD/OCRR/OPF-P 分层优化
板级端到端	有	非重点	KV260 DDR image demo
性能解释	较少	较少	stage/pass/counter 诊断

续提高 raw-HWC replay 吞吐并细化 pass 内部调度。第二，OFM debug packet 以每字节一个带地址 packet 的方式输出，便于验证但会增加 DMA 与软件重排开销，后续应设计连续 HWC burst 写回。第三，OCRR raw-HWC cache 对层形状敏感，Conv3 实验说明该路径更适合后端小空间尺寸层，后续可由 runtime 根据层形状自动选择。第四，LASA 当前主要覆盖以 1x1/3x3 标准卷积、逐层激活和池化为主的顺序型网络，对 depthwise/group/residual/concat 等结构仍需扩展。第五，column-level PSUM 仍处于探索阶段，需要完整 top-level xsim 和板级 A/B 后才能作为正式优化。

此外，补充功耗实测和更广泛模型验证也有助于提高结论的可比性和可推广性。功耗部分可结合 Vivado power estimate 与外部功耗计结果进行交叉验证；模型部分可优先选择同样以 1x1/3x3 标准卷积为主的轻量网络，评估 LASA 层自适应机制和 KCS/BSD/OCRR/OPF-P 优化策略在不同层形状下的适用范围。

6.8 本章小结

本章给出了 LASA 加速器的正确性、性能、资源和实验分析。实验表明，本文设计能够在 KV260/XCK26 上完成真实 Conv0–Conv9 链式推理，并通过 BSD、OCRR 和 OPF-P 将端到端 DDR demo 延迟降低到约 0.29 s。性能计数器进一步说明，优化后瓶颈已从 PSUM drain 和权重加载转移到 feeder/replay 与 compute idle，为后续工作提供了明确方向。

参考文献

- [1] REDMON J, DIVVALA S, GIRSHICK R, You only look once: Unified, real-time object detection// Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 779-788.
- [2] REDMON J FARHADI A. YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767, 2018.
- [3] QIU J, WANG J, YAO S, Going deeper with embedded FPGA platform for convolutional neural network//Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2016: 26-35.
- [4] GUO K, SUI L, QIU J, Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2019, 37(1): 35-47.
- [5] KUNG H T. Why systolic architectures?//Computer: vol. 15: 1. 1982: 37-46.
- [6] JOUPPI N P, YOUNG C, PATIL N, In-datacenter performance analysis of a tensor processing unit// Proceedings of the 44th Annual International Symposium on Computer Architecture. 2017: 1-12.
- [7] CHEN Y H, KRISHNA T, EMER J S, Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. ACM SIGARCH Computer Architecture News, 2016, 44(3): 367-379.
- [8] CHEN Y H, YANG T J, EMER J, Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. IEEE Journal on Emerging and Selected Topics in Circuits and Systems, 2019, 9(2): 292-308.
- [9] SZE V, CHEN Y H, YANG T J, Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE, 2017, 105(12): 2295-2329.
- [10] WEI X, YU C H, ZHANG P, Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs//Proceedings of the 54th Annual Design Automation Conference. 2017: 1-6.
- [11] MA Y, CAO Y, VRUDHULA S, Scalable and modularized RTL compilation of convolutional neural networks onto FPGA//2018 28th International Conference on Field Programmable Logic and Applications. 2018: 1-8.
- [12] DING C, WANG S, LIU L, A lightweight FPGA-based CNN accelerator design for YOLOv3-tiny. Microelectronics Journal, 2022, 120: 105344.
- [13] JACOB B, KLIGYS S, CHEN B, Quantization and training of neural networks for efficient integer-arithmetic-only inference. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018: 2704-2713.
- [14] Arm Ltd.AMBA AXI and ACE Protocol Specification. Arm Ltd., 2021.
- [15] Xilinx Inc.Zynq UltraScale+ MPSoC Data Sheet: DC and AC Switching Characteristics. Xilinx Inc., 2022.

- [16] CHEN X, LI J, ZHAO Y. Hardware Resource and Computational Density Efficient CNN Accelerator Design Based on FPGA//2021 IEEE International Conference on Integrated Circuits, Technologies and Applications (ICTA). Zhuhai, China, 2021.